

Data Structure : Assignment 1B

1. Explain the different types of data structures with examples.

Data structures are categorized into two main types: **Linear** and **Non-linear**.

- **Linear Data Structures:** Elements are arranged in a sequence.
 - **Array:** Fixed size, stores elements of the same type.
Example: `arr = [1, 2, 3, 4]`
 - **List (Python):** Dynamic array. Example: `my_list = ["apple", "banana"]`
 - **Stack:** Follows LIFO (Last In, First Out).
 - **Queue:** Follows FIFO (First In, First Out).
- **Non-linear Data Structures:** Elements are arranged hierarchically or in graphs.
 - **Tree:** Hierarchical structure (e.g., Binary Tree).
 - **Graph:** Set of nodes connected by edges.
 - **Heap:** A specialized tree-based structure satisfying the heap property.

2. What is recursion? How does it work? Explain with an example of a recursive function in Python.

Recursion is a technique where a function calls itself to solve smaller instances of the problem.

It works with two main components:

- **Base Case:** The stopping condition.
- **Recursive Case:** The function calls itself.

Example:

```
python Copy Edit

def factorial(n):
    if n == 0:
        return 1 # base case
    return n * factorial(n - 1) # recursive case

print(factorial(5)) # Output: 120
```

3. Explain the concept of time and space complexity with examples.

- **Time Complexity:** Measures the time taken as a function of input size n .
 - Example: $O(n)$ for a linear search, where the algorithm checks each item once.
 - **Space Complexity:** Measures the memory used relative to input size.
 - Example: A function storing all results in a list uses $O(n)$ space.
-

4. Differentiate between Big O, Theta (Θ), and Omega (Ω) notations with suitable examples.

- **Big O (O):** Worst-case scenario.
Example: Linear search is $O(n)$.
 - **Theta (Θ):** Average-case or tight bound.
Example: Balanced binary search has $\Theta(\log n)$.
 - **Omega (Ω):** Best-case scenario.
Example: Linear search may find the target at first index $\rightarrow \Omega(1)$.
-

5. Describe the importance of efficient memory utilization in data structures and how Python manages memory.

Efficient memory usage:

- Improves performance
- Reduces overhead
- Prevents memory leaks

Python Memory Management:

- Uses automatic garbage collection.
- Objects are managed by reference counting.

- Larger objects like lists are stored in heap memory.

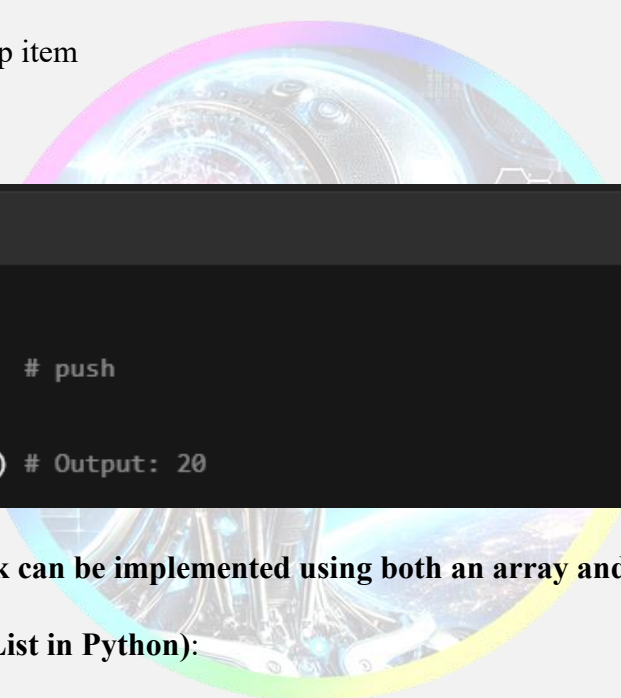
6. What is a stack? Explain its working principle with an example in Python.

A **stack** is a LIFO (Last In, First Out) data structure.

Operations:

- `push()`: Add item
- `pop()`: Remove item
- `peek()`: View top item

Example:



```
python                                                                    Copy Edit

stack = []
stack.append(10)    # push
stack.append(20)
print(stack.pop()) # Output: 20
```

7. Explain how a stack can be implemented using both an array and a linked list in Python.

- Using Array (List in Python):

```
stack = []
stack.append(1)
stack.pop()
```

Using Linked List:

```
class Node:
```

```
    def __init__(self, data):
```

```
        self.data = data
```

```
        self.next = None
```

```
class Stack:
```

```
    def __init__(self):
```

```
        self.top = None
```

```
    def push(self, value):
```

```
        new_node = Node(value)
```

```
        new_node.next = self.top
```

```
        self.top = new_node
```

```
    def pop(self):
```

```
        if self.top is None:
```

```
            return None
```

```
        value = self.top.data
```

```
        self.top = self.top.next
```

```
        return value
```



8. Describe real-life applications of stacks and how they are used in function calls and undo/redo operations.

- **Function Calls:** Uses call stack to manage local variables and return addresses.
 - **Undo/Redo:** Undo pushes actions onto a stack, redo uses another stack to reverse undo.
 - **Backtracking problems:** Like maze solving, recursion, etc.
 - **Browser history:** Uses a stack to go back to the previous page.
-

9. Explain how to convert an infix expression to postfix using a stack with an example.

Steps:

1. Use a stack for operators.
2. Output operands immediately.
3. Apply precedence rules for operators.

Example: Infix: $A + B * C$

Postfix: $A B C * +$

Algorithm: Use the Shunting Yard algorithm by Dijkstra.

10. Describe the process of evaluating a postfix expression using a stack. Provide a Python implementation.

Steps:

1. Scan from left to right.
2. Push operands onto the stack.
3. On operator, pop two items, apply operation, push result.

Python Example:

```
def evaluate_postfix(expr):
    stack = []
    for token in expr.split():
        if token.isdigit():
            stack.append(int(token))
        else:
            b = stack.pop()
            a = stack.pop()
            if token == '+':
                stack.append(a + b)
            elif token == '*':
                stack.append(a * b)
    return stack[0]

print(evaluate_postfix("2 3 4 * +")) # Output: 14
```

11. What is a queue? Explain how it follows the FIFO (First-In, First-Out) principle.

A **queue** is a linear data structure that follows the **FIFO** principle, meaning the first element added is the first to be removed.

Operations:

- enqueue(): Add to rear
- dequeue(): Remove from front

Example in Python:

```
from collections import deque

queue = deque()

queue.append(1) # enqueue

queue.append(2)

print(queue.popleft()) # dequeue → 1
```

12. Explain the differences between simple queue, circular queue, and double-ended queue (deque).

- **Simple Queue:** Insertion at rear, deletion from front. FIFO structure.

- **Circular Queue:** Last position connects to the first to reuse space efficiently.
- **Deque (Double-Ended Queue):** Insertion and deletion at both ends.

Type	Insert	Delete
Simple Queue	Rear	Front
Circular Queue	Rear (wraps)	Front (wraps)
Deque	Both ends	Both ends

13. Discuss the applications of queues in computer science and real life.

Applications:

- **CPU Scheduling:** Processes are queued by priority or time slice.
- **IO Buffers:** Queues manage data flow.
- **Printers:** Jobs are printed in the order they're queued.
- **Call centers:** Customers wait in queue for agents.
- **BFS Traversal:** Uses queue for visiting graph nodes level by level.

14. How is a priority queue different from a normal queue? Explain how a priority queue is implemented using a heap.

- **Normal Queue:** FIFO—removal is based on order of arrival.
- **Priority Queue:** Elements are removed based on priority, not arrival time.

Heap Implementation (Min-Heap or Max-Heap):

```
import heapq
```

```
pq = []
```

```
heapq.heappush(pq, (1, "low"))
```

```
heapq.heappush(pq, (0, "high"))  
  
print(heapq.heappop(pq)) # Output: (0, 'high')
```

15. Write a Python program to implement a circular queue using an array.

```
class CircularQueue:
```

```
    def __init__(self, size):
```

```
        self.queue = [None]*size
```

```
        self.max_size = size
```

```
        self.front = self.rear = -1
```

```
    def enqueue(self, data):
```

```
        if (self.rear + 1) % self.max_size == self.front:
```

```
            print("Queue is Full")
```

```
            return
```

```
        elif self.front == -1: # First element
```

```
            self.front = self.rear = 0
```

```
        else:
```

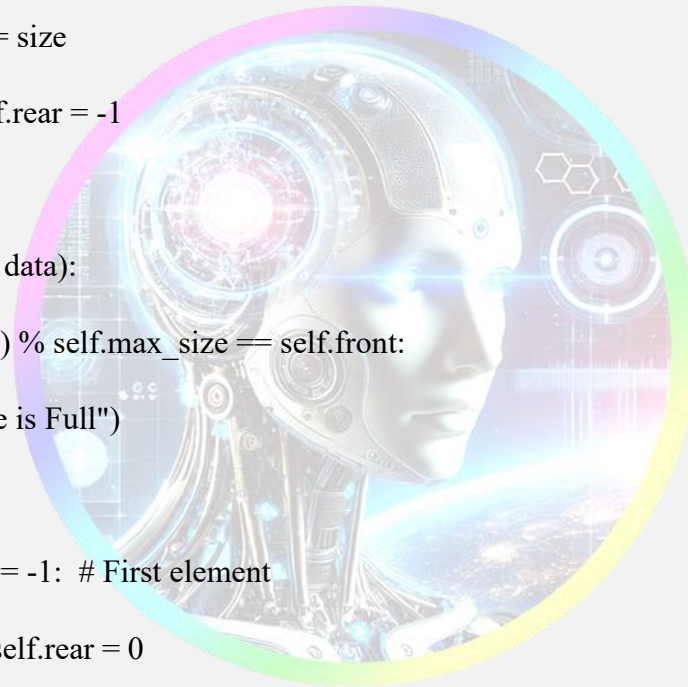
```
            self.rear = (self.rear + 1) % self.max_size
```

```
            self.queue[self.rear] = data
```

```
    def dequeue(self):
```

```
        if self.front == -1:
```

```
            print("Queue is Empty")
```



```
    return

elif self.front == self.rear:

    temp = self.queue[self.front]

    self.front = self.rear = -1

else:

    temp = self.queue[self.front]

    self.front = (self.front + 1) % self.max_size

return temp
```

16. What is a double-ended queue (deque)? Explain its operations and advantages over a normal queue.

A deque allows insertion and deletion from both ends: front and rear.

Operations:

- `append()`: Add to right
- `appendleft()`: Add to left
- `pop()`: Remove from right
- `popleft()`: Remove from left

Advantages:

- More flexible than queues.
- Useful for problems like sliding window, palindrome checking, etc.

17. Describe the implementation of multiple stacks and queues using a single array.

Concept:

- Divide the array logically.
- Use index manipulation to manage multiple stacks/queues.

Example (2 stacks in one array):

```
class TwoStacks:  
    def __init__(self, size):  
        self.arr = [None]*size  
        self.top1 = -1  
        self.top2 = size  
  
    def push1(self, val):  
        if self.top1 < self.top2 - 1:  
            self.top1 += 1  
            self.arr[self.top1] = val  
  
    def push2(self, val):  
        if self.top1 < self.top2 - 1:  
            self.top2 -= 1  
            self.arr[self.top2] = val
```

18. Explain how breadth-first search (BFS) algorithm uses queues for graph traversal.

BFS explores all neighbors of a node before going to the next level, and uses a queue to track the order of visiting.

Steps:

1. Enqueue starting node
2. Dequeue a node, mark as visited
3. Enqueue all unvisited neighbors

Example:

```

from collections import deque

def bfs(graph, start):
    visited = set()
    queue = deque([start])
    while queue:
        node = queue.popleft()
        if node not in visited:
            print(node)
            visited.add(node)
            queue.extend(graph[node])

```

19. Compare stacks and queues based on their operations, use cases, and performance.

Feature	Stack	Queue
Order	LIFO	FIFO
Operations	push, pop	enqueue, dequeue
Use Cases	Undo, recursion	Scheduling, BFS
Performance	O(1) for main ops	O(1) for main ops

Both are linear, but stack is better for backtracking, and queue is better for ordering tasks.

20. What are the advantages and disadvantages of using a linked list to implement a queue compared to using an array?

Linked List Queue:

- **Advantages:**
 - Dynamic size (no overflow unless memory full).
 - O(1) enqueue and dequeue.
- **Disadvantages:**

- Uses extra memory for pointers.
- Slightly slower due to pointer dereferencing.

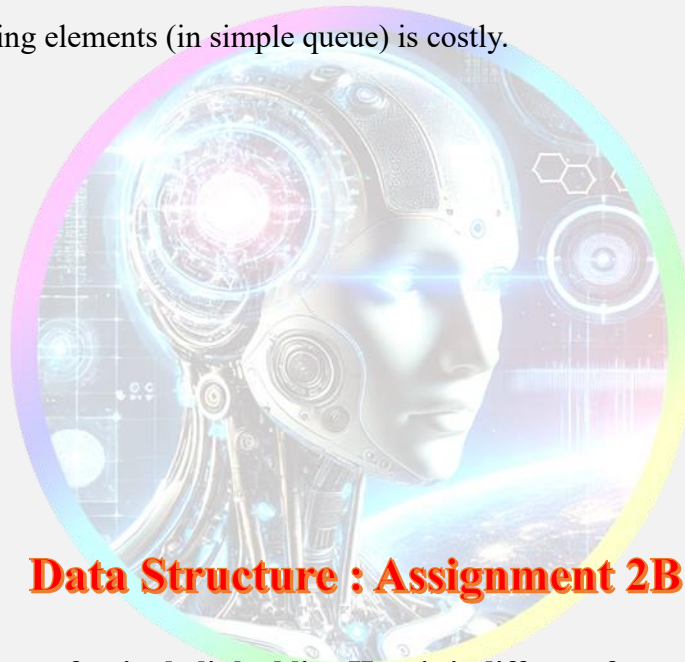
Array Queue:

- **Advantages:**

- Faster access time (direct index access).

- **Disadvantages:**

- Fixed size unless using dynamic resizing.
- Shifting elements (in simple queue) is costly.



Data Structure : Assignment 2B

1. Explain the concept of a singly linked list. How is it different from an array? Discuss its advantages and disadvantages.

A **singly linked list** is a linear data structure where each node contains **data** and a **reference to the next node**. It doesn't use contiguous memory.

Differences from Array:

- Linked list is **dynamic**; array has **fixed size**.
- Insertions/deletions are **faster** in linked lists ($O(1)$ at head).
- Arrays allow **random access**; linked lists do not.

Advantages:

- Dynamic size
- Efficient insertions/deletions

Disadvantages:

- No direct access
- Extra memory for pointers

2. Describe the process of inserting a node at the beginning, middle, and end of a singly linked list. Provide Python code for each operation.

```
Linked list > practice.py > ...
1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.next = None
5
6 class SinglyLinkedList:
7     def __init__(self):
8         self.head = None
9
10    def insert_at_beginning(self, data):
11        new_node = Node(data)
12        new_node.next = self.head
13        self.head = new_node
14
15    def insert_after_node(self, prev_node, data):
16        if not prev_node:
17            print("Previous node not found.")
18            return
19        new_node = Node(data)
20        new_node.next = prev_node.next
21        prev_node.next = new_node
22
```

```
23    def insert_at_end(self, data):
24        new_node = Node(data)
25        if not self.head:
26            self.head = new_node
27            return
28        last = self.head
29        while last.next:
30            last = last.next
31        last.next = new_node
32
33    def display(self):
34        current = self.head
35        while current:
36            print(current.data, end=" -> ")
37            current = current.next
38        print("None")
39
40    # Example Usage
41    ll = SinglyLinkedList()
42
```

```

43 # Insert at beginning
44 ll.insert_at_beginning(10)
45 ll.insert_at_beginning(5)
46
47 # Insert at end
48 ll.insert_at_end(20)
49
50 # Insert after first node (after 5)
51 ll.insert_after_node(ll.head, 7)
52
53 # Display the linked list
54 ll.display()
55

```

3. How do you reverse a singly linked list? Explain the algorithm and write Python code to implement it.

Algorithm:

1. Initialize prev = None, curr = head
2. Traverse list: change curr.next to prev
3. Move prev and curr forward

def reverse_list(head):

prev = None

curr = head

while curr:

next_node = curr.next

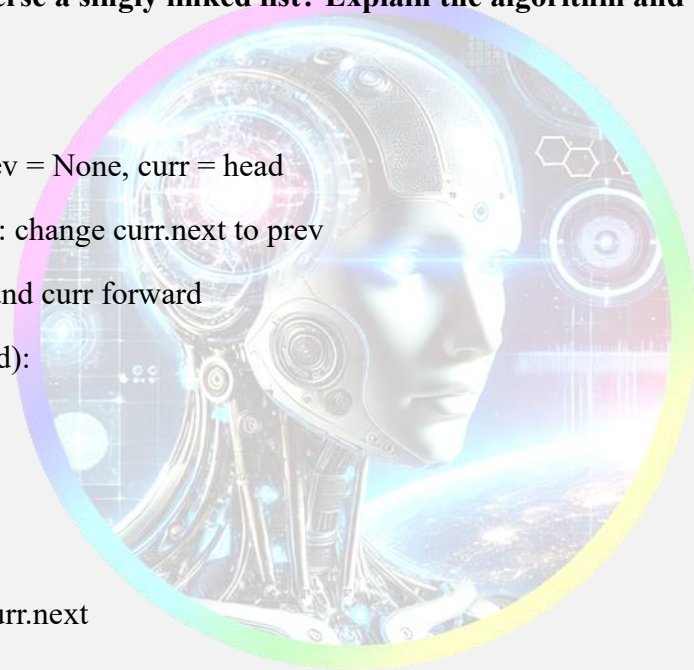
curr.next = prev

prev = curr

curr = next_node

return prev

- 4. What is the time complexity of various operations (insertion, deletion, searching, traversal) in a singly linked list? Justify your answer.**



Operation	Time Complexity	Reason
Insertion	O(1) at head, O(n) at end/middle	Traverse needed for non-head
Deletion	O(1) at head, O(n) at position	Need to find node
Searching	O(n)	Linear search
Traversal	O(n)	Visit every node

5. Explain how memory is allocated and managed in a singly linked list. How does it differ from contiguous memory allocation in arrays?

- Linked lists use **non-contiguous memory**; each node is created dynamically.
- Arrays allocate a **block of contiguous memory**, leading to resizing costs.
- Linked lists don't waste space but use **extra memory for pointers**.

6. How is a stack implemented using a linked list? Explain the push and pop operations with Python code.

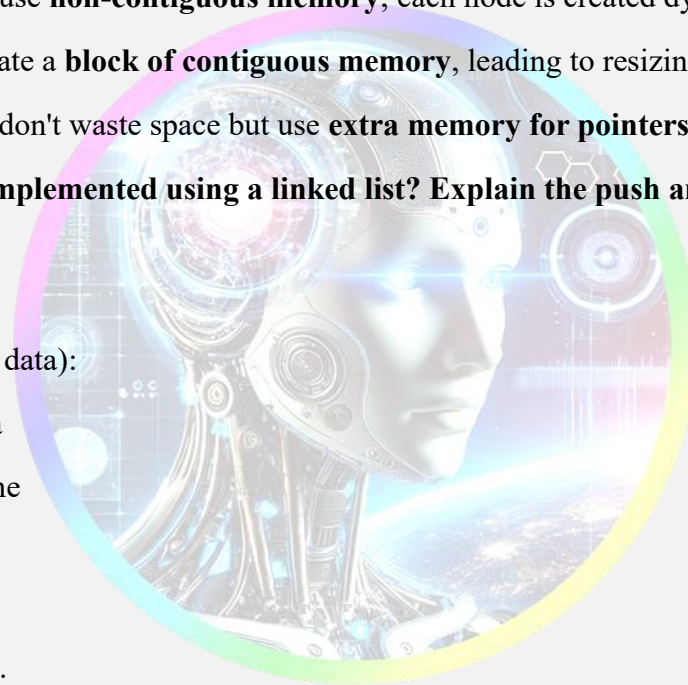
```
class StackNode:
```

```
    def __init__(self, data):
        self.data = data
        self.next = None
```

```
class Stack:
```

```
    def __init__(self):
        self.top = None
```

```
    def push(self, data):
        node = StackNode(data)
        node.next = self.top
        self.top = node
```



```
def pop(self):  
    if not self.top:  
        return None  
  
    data = self.top.data  
    self.top = self.top.next  
    return data
```

7. What are the advantages of using a linked list to implement a stack over an array? Discuss with examples.

Advantages:

- Dynamic size: No overflow unless memory full
- No resizing cost like arrays

Example: If many unknown operations are performed, a linked list avoids reallocation issues present in arrays.

8. Explain how a queue is implemented using a linked list. Provide Python code for enqueue and dequeue operations.

class QueueNode:

```
def __init__(self, data):  
    self.data = data  
    self.next = None
```

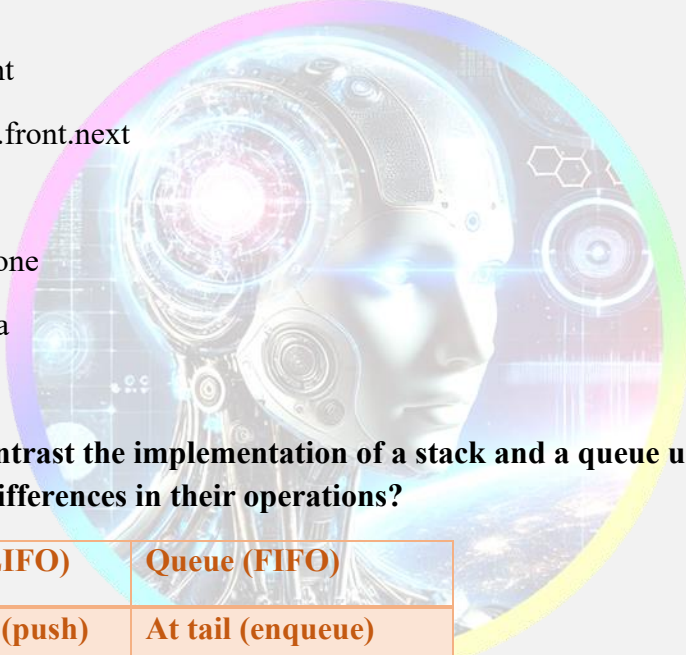
class Queue:

```
def __init__(self):  
    self.front = self.rear = None  
  
def enqueue(self, data):  
    node = QueueNode(data)
```

```
if not self.rear:
    self.front = self.rear = node
    return
self.rear.next = node
self.rear = node
```

```
def dequeue(self):
    if not self.front:
        return None
    temp = self.front
    self.front = self.front.next
    if not self.front:
        self.rear = None
    return temp.data
```

9. Compare and contrast the implementation of a stack and a queue using a linked list. What are the key differences in their operations?



Feature	Stack (LIFO)	Queue (FIFO)
Insert	At head (push)	At tail (enqueue)
Delete	From head (pop)	From head (dequeue)
Use	Recursion, Undo	BFS, Scheduling

Main difference: Stack operates only at one end, Queue at both ends (insert rear, remove front).

10. What is the time complexity of stack and queue operations when implemented using a linked list? Justify your answer.

All basic operations in both stack and queue (insert, remove) are:

- Time Complexity: $O(1)$

- Justification: Insertion and deletion are done at head or tail using pointer updates, no traversal needed.

11. Explain how polynomials can be represented using linked lists. Provide an example and write Python code to create a polynomial.

Each node in the linked list stores a **coefficient** and an **exponent**. The list is usually sorted in descending order of exponents.

Example: $5x^3 + 2x^2 + 3$ can be represented as:

```
python
class PolyNode:
    def __init__(self, coeff, expo):
        self.coeff = coeff
        self.expo = expo
        self.next = None

def create_polynomial(terms):
    head = None
    for coeff, expo in sorted(terms, key=lambda x: -x[1]):
        new_node = PolyNode(coeff, expo)
        new_node.next = head
        head = new_node
    return head
```

12. Describe the process of adding two polynomials using linked lists. Explain the algorithm and provide Python code for the same.

Algorithm:

- Traverse both lists simultaneously.
- If exponents match: add coefficients.
- Else: copy the node with the higher exponent.

Code:

```
def add_polynomials(poly1, poly2):
    dummy = PolyNode(0, 0)
    tail = dummy
```

```

while poly1 and poly2:
    if poly1.expo == poly2.expo:
        tail.next = PolyNode(poly1.coeff + poly2.coeff, poly1.expo)
        poly1, poly2 = poly1.next, poly2.next
    elif poly1.expo > poly2.expo:
        tail.next = PolyNode(poly1.coeff, poly1.expo)
        poly1 = poly1.next
    else:
        tail.next = PolyNode(poly2.coeff, poly2.expo)
        poly2 = poly2.next
    tail = tail.next

tail.next = poly1 or poly2
return dummy.next

```

13. What is the time complexity of polynomial addition using linked lists? Discuss the steps involved in the process.

- **Time Complexity:** $O(m + n)$ where m and n are the number of terms in each polynomial.
- **Steps:**
 1. Traverse both polynomials once.
 2. Compare exponents.
 3. Merge nodes accordingly.

14. How do you handle exponents and coefficients while adding two polynomials using linked lists? Explain with an example.

- **Matching exponents:** add coefficients.
- **Different exponents:** keep the node with the higher exponent.

Example:

- $3x^2 + 2x + 1$ and $4x^2 + x \rightarrow 7x^2 + 3x + 1$

Handled in code by checking:

```
if p1.expo == p2.expo:
```

```
    result_coeff = p1.coeff + p2.coeff
```

15. What are the advantages of using linked lists for polynomial addition over arrays? Discuss with examples.

Advantages:

- No need for sparse storage; zero coefficients are skipped.
- Efficient insertion/deletion of terms.
- Handles large exponents without fixed-size arrays.

Example: Adding $x^{100} + 2x^5 + 3$ using an array requires size 101; linked list only needs 3 nodes.

16. What is a doubly linked list? How is it different from a singly linked list? Discuss its advantages and disadvantages.

A doubly linked list (DLL) has nodes with **three parts**: prev, data, and next.

Differences from SLL:

- DLL can be traversed both ways (forward and backward).
- Requires more memory (extra pointer).

Advantages:

- Easy backward traversal.
- Efficient deletion without needing previous node.

Disadvantages:

- More memory usage.
 - Complex implementation.
-

17. Explain the process of inserting a node at the beginning, middle, and end of a doubly linked list. Provide Python code for each operation.

```
class DNode:
    def __init__(self, data):
        self.data = data
        self.prev = self.next = None

# Insert at beginning
def insert_begin(head, data):
    node = DNode(data)
    node.next = head
    if head:
        head.prev = node
    return node

# Insert at end
def insert_end(head, data):
    node = DNode(data)
    if not head:
        return node
    temp = head
    while temp.next:
        temp = temp.next
    temp.next = node
    node.prev = temp
    return head
```

```
# Insert after position
def insert_after(head, pos, data):
    temp = head
    for _ in range(pos):
        temp = temp.next
    node = DNode(data)
    node.next = temp.next
    node.prev = temp
    if temp.next:
        temp.next.prev = node
    temp.next = node
    return head
```

18. How do you reverse a doubly linked list? Explain the algorithm and write Python code to implement it.

Algorithm:

- Swap next and prev of each node.
- Update head to the last node.

Python Code:

```
def reverse_dll(head):
```

```
    temp = None
```

```
    current = head
```

```
    while current:
```

```
        temp = current.prev
```

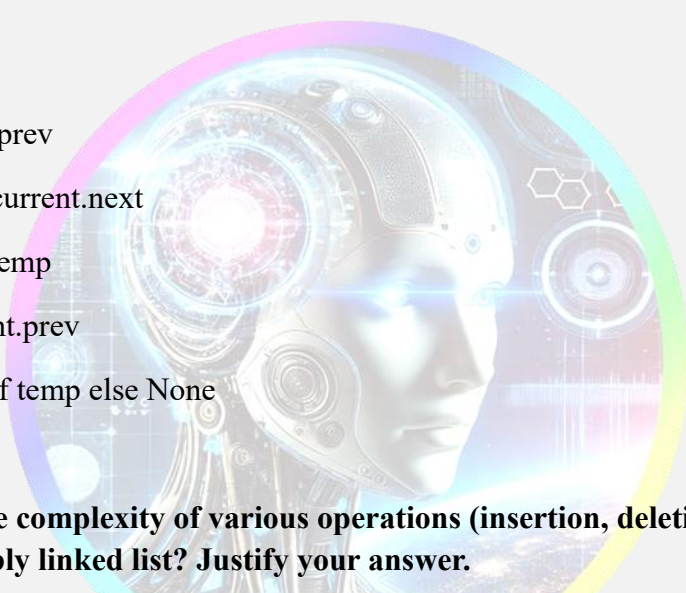
```
        current.prev = current.next
```

```
        current.next = temp
```

```
        current = current.prev
```

```
    return temp.prev if temp else None
```

19. What is the time complexity of various operations (insertion, deletion, searching, traversal) in a doubly linked list? Justify your answer.



Operation	Time Complexity	Reason
Insertion	O(1) (if position known)	Adjust 2 pointers
Deletion	O(1) (if node given)	Use prev and next
Searching	O(n)	Linear traversal needed
Traversal	O(n)	Visit all nodes

20. Compare and contrast singly linked lists and doubly linked lists. When would you prefer one over the other?

Feature	Singly Linked List	Doubly Linked List
Memory Usage	Less (1 pointer)	More (2 pointers)
Traversal	One-direction	Bi-directional
Deletion	$O(n)$ if no previous node	$O(1)$ using prev pointer
Preferred when...	Memory is tight, simple use	Efficient deletion/traversal

